BardBox — Multi-Tenant White-Label Platform

Overview

This is the core product built by **BardBox** (Performance Marketing department). It is a white-label SaaS platform that provisions client panels without any coding and without creating any new deployments. One frontend, one backend, one database cluster — serves every role and every client.

Deployment Reality

```
Repos        : 2 only (bizcivitas-performance-marketing + pm-backend)
Deployments  : 2 only (one frontend server + one backend server)
DNS          : 1 wildcard record  *.bardbox.com → same server
Work per new client : ZERO — fully automated via internal API
```

No new repo. No new deployment. No manual DNS entry. Client exists the moment Setup Wizard hits Launch.

Architecture

```
BardBox (CEO)
    └── Revenue & Profit overview only. No operational access.

Super Admin (BardBox Operations)
    └── Creates and manages departments.
    └── Assigns which features each department can offer to clients.

Departments (Performance Marketing / Social Media / Data Analyst)
    └── Each department logs in separately.
    └── Runs Setup Wizard to onboard new clients.
    └── Manages only their own clients.

Clients
    └── Log into the SAME frontend at their own subdomain
(clientb.bardbox.com)
    └── See ONLY their assigned features — nothing else.
    └── Get a unique tracking script to embed on their landing page.
    └── Have no visibility into BardBox, Super Admin, or other clients.
```

Role Hierarchy

Role	Access	Responsibilities
BardBox CEO	Revenue dashboard only	See profit, revenue per department, growth
Super Admin	Full operational control	Create departments, assign features, manage all
Department	Their clients only	Setup Wizard, onboard clients, manage billing
Client	Their subdomain panel only	Use features given by their department

---

How One Frontend Serves All Clients — Subdomain Routing

Every client gets a subdomain under `bardbox.com`. All subdomains point to the same server via one wildcard DNS record.

```
DNS (one-time setup, never touch again):
    *.bardbox.com  →  same server IP

Requests:
    bizcivitas.bardbox.com  →  Next.js middleware reads host header → slug =
"bizcivitas"
    clientb.bardbox.com     →  Next.js middleware reads host header → slug =
"clientb"
    clientc.bardbox.com     →  Next.js middleware reads host header → slug =
"clientc"
```

Next.js Middleware (automatic, runs on every request)

```
Request hits clientb.bardbox.com/admin
        │
        └── middleware.ts
                │
                ├── Extract slug from host → "clientb"
                ├── GET /api/internal/client-config?slug=clientb
                │       └── pm-backend reads master_db.clients
                │               └── returns { branding, features, db,
integrations }
                │
                └── Injects config into request context
                        └── /admin layout reads config
                                ├── Applies branding (logo, colors, app name)
                                └── Shows only features[] assigned to this
client
```

No code change. No redeployment. New client = new record in DB = panel works immediately.

System Flow

```
1. Super Admin logs in
        │
        └── Creates department (e.g. Performance Marketing)
                └── Assigns master feature list to that department
                        └── PM gets: [email, whatsapp, members, forms,
                                      scoring, kanban, templates, plans,
                                      conversations, inquiries, tracking]

2. Department logs in (Performance Marketing)
        │
        └── Runs Setup Wizard for new client
                Step 1 → Client Info        (name, slug, contact email)
                Step 2 → Branding           (logo, primary color, app name,
favicon)
                Step 3 → Feature Select     (only from what Super Admin
allowed)
                Step 4 → Billing            (tier, price, expiry date)
                Step 5 → Review & Launch
                        │
                        └── Internal API chain runs automatically:
                                ├── POST /internal/provision    → stores
client in master_db
                                ├── POST /internal/create-db    → creates
clientb_db in Atlas, seeds it
                                ├── POST /internal/issue-jwt    → signs RS256
```

```
license key, stores hashed
                                ├── POST /internal/gen-tracking → generates
unique tracking key
                                └── POST /internal/send-email   → emails
credentials to client

3. Client receives email
        │
        └── Panel URL      → clientb.bardbox.com
            Email          → admin@clientb.com
            Password       → [temp, must change on first login]
            Tracking Code  → unique <script> tag for their landing page

4. Client logs in at clientb.bardbox.com
        └── Middleware detects slug → fetches config from DB
            Sees ONLY their white-labelled panel
            with ONLY their assigned features
            No visibility into BardBox, Super Admin, other clients, or other
departments
```

Client Provisioning — Internal API Chain

Same pattern as template creation/fetching already built in the system. Backend API drives everything automatically when Setup Wizard submits.

```
POST /internal/provision
    Body: { name, slug, contactEmail, branding, features, tier, billing }
    Action: Creates client record in master_db.clients

POST /internal/create-db
    Body: { slug }
    Action: Creates {slug}_db in Atlas cluster
            Seeds collections: members, events, inquiries, conversations,
                               payments, plans, settings, roles

POST /internal/issue-jwt
    Body: { slug, db, features, tier, exp }
    Action: Signs JWT with BardBox RS256 private key
            Stores hashed in master_db.clients.jwtHash
            Returns raw token (used by backend middleware per request)

POST /internal/gen-tracking
    Body: { slug }
    Action: Generates unique tracking key
            Stores in master_db.clients.trackingKey
```

```
POST /internal/send-email
    Body: { contactEmail, slug, tempPassword, trackingScript }
    Action: Sends credential email via Nodemailer
```

Tracking Script System

BardBox does not host or build landing pages for clients. Instead, we give each client a **unique tracking script** they embed on any landing page they own — built on any platform (WordPress, Webflow, custom HTML, anything).

What Client Gets

```
<!-- BardBox Tracking — paste in <head> of your landing page -->
<script
  src="https://track.thebardbox.com/sdk.js"
  data-client="bizcivitas"
  data-key="eyJhbGciOiJSUzI1NiJ9...">
</script>
```

One line. Client pastes it in `<head>`. Done.

What the Script Tracks Automatically

```
sdk.js captures:
    ├── Page views + unique visitors
    ├── Button clicks (any CTA)
    ├── Form submissions → sent as leads to client's admin panel
    ├── Time on page
    ├── UTM parameters (source, medium, campaign, keyword)
    ├── Device type (mobile / desktop)
    └── Conversion events
```

Lead Capture (Zero Code from Client)

Client adds `data-bardbox-form="lead"` to any form on their page:

```
<form data-bardbox-form="lead">
    <input name="name" placeholder="Your Name" />
    <input name="phone" placeholder="Phone" />
    <input name="email" placeholder="Email" />
```

```
    <button type="submit">Join Now</button>
</form>
```

Script intercepts submit → sends to pm-backend collect endpoint → appears in client's Inquiries instantly.

Lead Flow

```
Visitor submits form on client's landing page (anywhere)
        │
        └── sdk.js → POST track.thebardbox.com/collect/bizcivitas
                        │
                        └── pm-backend reads slug → connects to bizcivitas_db
                                │
                                └── Writes to bizcivitas_db.inquiries
                                        │
                                        ▼
                        bizcivitas.bardbox.com/admin/inquiries
                        WhatsApp + Email notification triggered automatically
```

Tracking Code in Admin Panel

```
Admin Panel → Settings → Tracking Code
        ├── Script tag shown (copy button)
        ├── Installation instructions
        ├── Status: Active / Not Detected
        ├── Last seen: [timestamp]
        └── Analytics: views, clicks, leads this month
```

What Each Client Gets

```
Client (e.g. BizCivitas)
│
├── Subdomain Panel              bizcivitas.bardbox.com
│       ├── Inquiries    ← leads from tracking script appear here
│       ├── Members      ← converted leads become members
│       ├── Conversations
│       ├── Kanban       ← track lead through pipeline
│       ├── Forms
│       ├── Email & WhatsApp campaigns
│       ├── Scoring & Analytics
```

```
│       └── Plans & Billing
│       (Only features assigned by their department are visible)
│
└── Tracking Script              embed on any landing page
        └── Captures leads → flows into their Inquiries instantly
```

Everything white-labelled — client's logo, colors, app name applied at runtime from DB config. No BardBox branding visible.

Infrastructure

```
Repos (2 total — never changes regardless of client count)
    ├── bizcivitas-performance-marketing   ← Frontend (Next.js)
    └── pm-backend                         ← Backend (Express)

Deployments (2 total — never changes)
    ├── frontend.bardbox.com               ← serves ALL subdomains
    └── api.bardbox.com                    ← handles ALL client requests

DNS (1 wildcard — set once, never touch again)
    *.bardbox.com  →  frontend server IP

MongoDB Atlas (One Cluster)
    ├── master_db          ← BardBox internal (departments, clients, billing)
    ├── bizcivitas_db      ← BizCivitas client data
    ├── clientb_db         ← Client B data (created by Setup Wizard)
    └── clientc_db         ← Client C data (created by Setup Wizard) ...
```

JWT License Key

Each client's JWT is stored **in master_db** (not as env var — one backend handles all clients). The backend reads it per request from the authenticated session.

```
{
  "iss": "bardbox-performance-marketing",
  "client": "bizcivitas",
  "db": "bizcivitas_db",
  "domain": "bizcivitas.bardbox.com",
  "tier": "standard",
  "features": ["email", "whatsapp", "members", "forms", "tracking"],
  "exp": "2027-01-01"
}
```

- Signed with BardBox private key (RS256) — only BardBox can issue or revoke
- Validated on every `/admin` request via middleware
- Expired or revoked = client session immediately blocked
- Super Admin can revoke, renew, or upgrade from the panel

Multi-Tenant Backend Middleware

One backend handles all clients. Every request to `/admin/*` goes through tenant middleware:

```
Request: GET bizcivitas.bardbox.com/admin/members
        │
        └── pm-backend tenant middleware
                │
                ├── Read slug from subdomain → "bizcivitas"
                ├── Verify JWT from Authorization header
                │       └── Confirms client = "bizcivitas", db =
"bizcivitas_db"
                ├── Open connection to bizcivitas_db (connection pooled,
reused)
                └── Pass db handle to route handler
                        └── Handler queries bizcivitas_db.members only
                            Client A can NEVER access Client B's DB
```

Client-Specific Integrations (stored in DB, not env vars)

Since one backend serves all clients, client API keys are stored encrypted in master_db and fetched per request:

```
{
  "slug": "bizcivitas",
  "integrations": {
```

```
      "whatsapp": { "apiKey": "encrypted:...", "phoneNumberId": "..." },
      "razorpay":  { "keyId": "encrypted:...", "keySecret": "encrypted:..." },
      "smtp":      { "host": "smtp.example.com", "user": "...", "pass":
  "encrypted:..." }
    }
  }
```

Department sets these during Setup Wizard Step 1 (Client Info). Client can update them from their Settings panel.

Feature List (Master)

Super Admin controls which features each department can offer. Departments control which features each client gets.

Feature	Description
`tracking`	Unique tracking script + lead capture SDK
`inquiries`	Inquiry management (leads from tracking script)
`members`	Member management
`conversations`	Unified conversation inbox
`kanban`	Kanban board for lead pipeline
`email`	Email campaigns and templates
`whatsapp`	WhatsApp messaging and automation
`forms`	Form builder
`scoring`	Engagement scoring system
`templates`	Content templates
`plans`	Membership plans
`stories`	Success stories module
`api-integrations`	Third-party API connections
`plugins`	Plugin marketplace

White-Label Config

Each client's branding is stored in master_db and fetched by Next.js middleware on every request. No code change or redeployment needed.

```
{
  "slug": "clientb",
  "branding": {
    "appName": "Client B Hub",
    "logo": "https://cdn.clientb.com/logo.png",
    "primaryColor": "#FF5500",
    "favicon": "https://cdn.clientb.com/favicon.ico"
  },
  "subdomain": "clientb.bardbox.com",
  "features": ["tracking", "email", "members", "kanban"],
  "status": "active",
  "tier": "standard",
  "expiresAt": "2027-01-01"
}
```

Route Structure

```
src/app/
│
├── middleware.ts                  ← reads subdomain on every request
│                                    fetches client config from pm-backend
│                                    injects into request context
│
├── ceo/                           ← BardBox CEO login
│   ├── page.tsx                     revenue + profit dashboard
│   └── layout.tsx                   CEO auth guard
│
├── super-admin/                   ← Super Admin login
│   ├── page.tsx                     overview (all depts, all clients)
│   ├── departments/
│   │   ├── page.tsx                 list all departments
│   │   ├── create/page.tsx          create dept + assign features
│   │   └── [id]/page.tsx            manage dept (suspend/edit)
│   ├── clients/
│   │   └── page.tsx                 all clients across all depts
│   ├── features/
│   │   └── page.tsx                 master feature list management
│   └── layout.tsx                   Super Admin auth guard
│
├── department/                    ← Department login
│   ├── page.tsx                     client list + dept revenue
│   ├── clients/
│   │   ├── page.tsx                 their clients only
│   │   ├── setup-wizard/page.tsx    onboard new client (5 steps)
│   │   └── [id]/page.tsx            manage client (suspend/renew/reset)
```

```
│   └── layout.tsx                    dept auth guard
│
├── admin/                          ← Client panel (all clients share this)
│   ├── inquiries/                    leads from tracking script
│   ├── members/                      converted leads / members
│   ├── conversations/
│   ├── kanban/
│   ├── email/
│   ├── whatsapp/
│   ├── forms/
│   ├── scoring/
│   ├── templates/
│   ├── plans/
│   ├── stories/
│   ├── api-integrations/
│   ├── settings/
│   │   └── tracking/                 tracking script display + stats
│   └── layout.tsx                  ← reads client config from middleware
context
│                                     shows only features[] assigned to this
client
│
└── api/
    ├── internal/                   ← Setup Wizard automation endpoints
    │   ├── provision/route.ts        create client record in master_db
    │   ├── create-db/route.ts        create + seed client DB in Atlas
    │   ├── issue-jwt/route.ts        sign RS256 license key
    │   ├── gen-tracking/route.ts     generate tracking key
    │   └── send-email/route.ts       email credentials to client
    │
    └── tracking/
        └── collect/[clientSlug]/   ← public endpoint, receives sdk.js events
                                      CORS open — accepts any domain
```

Tracking SDK Endpoint

```
POST https://track.thebardbox.com/collect/:clientSlug

Body:
{
  "event": "form_submit",
  "data": {
    "name": "John Doe",
    "phone": "9999999999",
    "email": "john@example.com"
  },
```

```
    "meta": {
      "page": "https://bizcivitas.in",
      "utm_source": "google",
      "utm_campaign": "may-offer",
      "device": "mobile"
    }
  }

  Response:
  { "success": true }
```

CORS open — accepts requests from any domain (client's landing page can be anywhere).

Database Schema (master_db)

departments

```
{
  "_id": "ObjectId",
  "name": "Performance Marketing",
  "slug": "performance-marketing",
  "email": "pm@bardbox.com",
  "passwordHash": "...",
  "features": ["tracking", "email", "whatsapp", "members", "forms"],
  "status": "active",
  "createdAt": "2026-05-05"
}
```

clients

```
{
  "_id": "ObjectId",
  "departmentId": "ObjectId",
  "name": "BizCivitas",
  "slug": "bizcivitas",
  "subdomain": "bizcivitas.bardbox.com",
  "contactEmail": "admin@bizcivitas.in",
  "branding": {
    "appName": "BizCivitas",
    "logo": "...",
    "primaryColor": "#000000",
    "favicon": "..."
  },
  "features": ["tracking", "email", "whatsapp", "members"],
```

```
  "db": "bizcivitas_db",
  "jwtHash": "hashed_license_key",
  "trackingKey": "eyJhbGci...",
  "integrations": {
    "whatsapp": { "apiKey": "encrypted:...", "phoneNumberId": "..." },
    "razorpay":  { "keyId": "encrypted:...", "keySecret": "encrypted:..." },
    "smtp":      { "host": "...", "user": "...", "pass": "encrypted:..." }
  },
  "tier": "standard",
  "billing": {
    "amount": 50000,
    "currency": "INR",
    "renewsAt": "2027-01-01"
  },
  "status": "active",
  "createdAt": "2026-05-05"
}
```

billing

```
{
  "_id": "ObjectId",
  "clientId": "ObjectId",
  "departmentId": "ObjectId",
  "amount": 50000,
  "currency": "INR",
  "status": "paid",
  "paidAt": "2026-05-05"
}
```

CEO Dashboard Data

CEO sees aggregated data only — pulled from billing collection:

```
Total Revenue    = sum of all paid billing records
Total Profit     = revenue - costs
Per Dept Revenue = grouped by departmentId
Active Clients   = count of clients with status: active
Growth %         = MoM comparison
```

Environment Variables

Only 2 env files ever. Never changes per client.

```
# ── Frontend (bizcivitas-performance-marketing) ──
NEXT_PUBLIC_API_URL=https://api.bardbox.com

# ── Backend (pm-backend) ──
MONGO_URI=mongodb+srv://...@cluster.mongodb.net
MONGO_MASTER_DB=master_db
BARDBOX_PRIVATE_KEY=-----BEGIN RSA PRIVATE KEY-----...
SMTP_HOST=smtp.resend.com
SMTP_USER=...
SMTP_PASS=...
```

Client-specific keys (WhatsApp, Razorpay, SMTP) are stored encrypted in master_db.clients.integrations and fetched by the backend per request.

Build Phases

Phase 1 — Core Infrastructure

- ☐ master_db schema setup (departments, clients, billing)
- ☐ JWT license service (RS256 sign/verify/revoke)
- ☐ Auth system (4 roles: CEO, Super Admin, Department, Client)
- ☐ Multi-tenant middleware (subdomain → client config → DB routing)
- ☐ Feature flag guard on `/admin` routes

Phase 2 — Tracking SDK

- ☐ `sdk.js` — public tracking script
- ☐ `/api/tracking/collect/:clientSlug` — event receiver endpoint (CORS open)
- ☐ Lead capture via `data-bardbox-form` attribute
- ☐ UTM + device metadata capture
- ☐ Admin panel → Settings → Tracking Code page

Phase 3 — Super Admin Panel

- ☐ Department CRUD
- ☐ Feature assignment to departments
- ☐ All clients overview
- ☐ JWT revoke / renew controls

Phase 4 — Department Panel + Setup Wizard

- ☐ Department login + dashboard

- ☐ Setup Wizard (5 steps)
- ☐ Internal API chain (provision → create-db → issue-jwt → gen-tracking → send-email)
- ☐ Client management (suspend, renew, reset password)

Phase 5 — White-Label Engine

- ☐ Next.js middleware — subdomain detection + config fetch
- ☐ Branding applied at runtime (logo, colors, app name)
- ☐ Feature sidebar renders only assigned features
- ☐ Wildcard DNS setup (*.bardbox.com)

Phase 6 — CEO Dashboard

- ☐ Revenue charts
- ☐ Profit tracking
- ☐ Department breakdown
- ☐ Growth metrics

Phase 7 — New Departments

- ☐ Social Media department + their product
- ☐ Data Analyst department + their product

Tech Stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript
Styling	Tailwind CSS v4
State	Redux Toolkit
Charts	Recharts
Auth	JWT (jsonwebtoken, RS256)
Multi-tenancy	Next.js middleware (subdomain detection)
Database	MongoDB Atlas
Email	Nodemailer / Resend
Deployment	Railway / Coolify

Credential Delivery (on client creation)

```
Email to client:

Subject: Your BardBox Platform is Ready

Your panel has been set up and is ready to use.

Panel URL      : https://clientb.bardbox.com
Email          : admin@clientb.com
Password       : [temporary — must change on first login]

Tracking Code  : paste this in <head> of your landing page
-----------------------------------------------
<script
  src="https://track.thebardbox.com/sdk.js"
  data-client="clientb"
  data-key="eyJhbGci...">
</script>
-----------------------------------------------

If you have any issues, contact your account manager.
```

Built by BardBox — Performance Marketing Department